Staff Attendance & Overtime — Final Tech Doc (Build Plan + Feature Inventory + Improvement Roadmap)

Status	**FINAL** — for execution (Parts 1–4) + Pendency for founder review (Part 5). Screen inventory verified against the full screenshot set; remaining minor assumptions listed in Appendix B.
Author	Tech Lead
Date	2026-06-13
Inputs	Existing-system screenshots (web — list, profile Summarised/Separate/Absent views, Edit Sign In-Out, Edit Absent, action menu, date picker), `attendance_service` + `organization_service` schema exports, PM product spec *"Overtime & Regular hours for Staff"* (2026-06-12)
Supersedes	`STAFF_ATTENDANCE_OVERTIME_TECH_SPEC.md` (full greenfield proposal — kept as reference for the pendency items)
Direction (PM-agreed)	Reuse the existing DB schema. Rebuild the existing web staff-attendance system in the new project at full parity. Attendance corrections stay admin-driven (staff reaches out → admin edits). The regularization/approval workflow moves to the future **HR application**. Further betterments are proposed in Part 5 as pendency for founder review.

How to read this document

- **Part 1** — audit of the existing schema: what we reuse as-is, what needs small improvements, what is genuinely missing.
- **Part 2** — the build plan: data, calculation rules, API, and platform work.
- **Part 3** — **complete screen & feature inventory**: every screen, component, pop-up, state, and behavior — the review checklist for founder & PM that nothing is missed.
- **Part 4** — improvements folded into this build because they cost almost nothing on top of parity.
- **Part 5** — the improvement roadmap (pendency): each proposal with its impact on product goodwill.

PART 1 — Existing Schema Audit

Source: `attendance_service` and `organization_service` schema exports (2026-06-12).

1.1 The headline

Most of what we designed already exists. The earlier greenfield spec proposed `school_attendance_payroll_settings` and `staff_attendance_session` — the live schema already covers the majority of both, plus an approval pattern (on absences) and a generic audit-log table we can extend. The build is therefore mostly **API + frontend work, not schema work.**

1.2 Tables reused as-is (no changes)

Table	What it gives us	Notes
`staff_attendance_record`	The raw punch log: `attendance_date`, `signed_in_at/by_code/method`, `signed_out_at/by_code/method`, GPS `*_location_lat/long`, `*_within_geofence`, `clock_pin_used`, `current/primary_classroom_code`, `is_expected_today`, `day_of_week`, `status`, `total_worked_minutes`, soft delete (`deleted_at/by`), full audit columns	**This is our session table.** Multiple rows per staff per day = multiple sessions (matches the Separate view). See §1.3 for the two columns it lacks.

Table	What it gives us	Notes
`staff_classroom_transfer`	Transfer history (`from/to_classroom_code` , `transferred_at/by_code/by_role` , `transfer_reason` , `staff_attendance_id`)	Powers the classroom column per session row.
`staff_absence_record`	Absences with `absence_date/from/to` , `absence_type` , `absence_reason` , `mark_absent_method` — **and a complete approval workflow:** `requires_approval` , `approval_status` , `approved_by_code` , `approved_at`	Powers the Absent tab/card. The approval fields are a ready-made in-house pattern for the future regularization flow (Part 5, R-3).
`attendance_settings`	`auto_sign_out_enabled` , `auto_sign_out_delay_minutes` , `gps_sign_in_enabled` , geofence/proximity limits, outside-hours rules — per school	Auto sign-out is **already a per-school toggle**. Half of the missing-punch configurability we wanted exists today.
`schools` (org service)	`time_zone` per school	All date-bucketing and "school day" boundaries resolve against this. No new timezone storage needed.
`childcare_schedules` (org service)	`day_of_week` , `start_time` , `end_time` , `half_day` , `schedule_type` per school	The school-hours / working-day calendar. Used for expected hours display and (Part 5) working-days-in-month payroll math.
`clock_in_pin` , `attendance_pin` , `attendance_qr_code` , `attendance_kiosk_config` , `attendance_sign_in_method` , `attendance_parent_sign_in_method_config`	All sign-in/out methods (PIN, QR, kiosk, GPS, app)	Unchanged.
`attendance_notification_settings` , `notification_templates` , `notification_subscribers` (org service)	Notification plumbing with role/user subscriber targeting	Reused now for sign-in/out notifications; reused later for Part 5 nudges — no new infra.
`generated_reports`	`report_type` , `format` , `download_url` , `filters_json` , `expires_at`	Powers report export (CSV today; Excel/PDF per PM spec — see §3, S-2).

1.3 Tables reused with small improvements

`staff_attendance_record` — 2 gaps

Gap	Why it matters	Cheapest fix
Cannot tell a real punch-out from a system auto-close. When auto sign-out fires at school end time, the record looks identical to a genuine sign-out. This is the root of the wrong-overtime edge case (staff stayed till 3:30, school ends 2:00).	Every downstream number silently treats fabricated time as real. Admins can't even *find* these records to correct them.	Write `signed_out_method = 'SYSTEM'` when auto sign-out closes a record (zero schema change if the column is unconstrained varchar; otherwise add the enum value). Optionally add one `close_type` column (`NORMAL / SYSTEM_CLOSED / ADMIN_CORRECTED`) for explicitness.
No optimistic locking. Two admins editing the same record (or edit racing the auto-close job) silently last-write-wins.	Corrections are exactly the records people fight over.	Add `version INT` column; `PUT` carries it; mismatch → 409 → client refreshes. One column, one check.

`staff_overtime_config` — the money knobs are missing

Already has: `calculation_method` (daily/weekly basis — already configurable, matching the PM spec's "daily threshold + optional weekly threshold"), `daily_regular_hours` , `weekly_regular_hours` , `is_active` , school/org scoping.

Missing column	Purpose	Default
`ot_multiplier`	OT pay = rate × multiplier; varies by location (the 1.5x)	`1.50`
`ot_min_daily_minutes`	Grace: OT below this per day is ignored	`15`
`ot_rounding_minutes`	Floor OT to unit so stray minutes don't move payroll	`15`
`undertime_min_daily_minutes` / `undertime_rounding_minutes`	Same knobs for shortfall	`15` / `15`
`undertime_deduction_factor`	Deduction at 1.0× or 0.5× of rate (admin policy)	`1.0`

~5 nullable columns with defaults — additive, zero migration risk. (Effective-dating of this config is deliberately deferred to Part 5, R-6.)

`attendance_settings_audit_log` — generalize its usage, not its schema

The table is already **generic**: `table_name`, `record_id`, `column_name`, `old_value`, `new_value`, `action`, `changed_by`, `changed_at`, `ip_address`, `user_agent`. Today it only receives settings changes. **Improvement: also write a row on every `staff_attendance_record` edit and delete.** No schema change at all — we get a full attendance audit trail (who corrected whose time, from what, to what) for free. This is the backbone of trustworthy admin corrections (§2.6).

`staff_attendance_summary` — keep for parity, mark for review

The scheduler-fed aggregate table: `summary_date`, `summary_type`, `regular_hours`, `overtime_hours`, `total_attended_minutes`, `total_break_minutes`, `days_worked`, `late_count`, `early_departure_*`, `first_sign_in_at`, `last_sign_out_at`, and notably `is_finalized`.

- **Now (parity):** keep reading it — the existing web UI is built on it and the PM wants like-for-like first.
- **Known weakness to manage:** when an admin edits/deletes a record *after* the summary job ran, the summary is stale until recomputed. Mitigation in this build (and per the PM spec's Gherkin: *"execute the function to calculate regular & overtime hrs after every sign-out entry"*): **every sign-out, edit, or delete triggers a targeted recompute of that staff-day's summary row** — corrections and punches reflect immediately. A contained fix, not a rearchitecture. |
- **Later (Part 5, R-1):** this table is the deprecation target — and its `is_finalized` flag is the seed of proper payroll locking (R-5).

1.4 Genuinely missing (deferred to the HR application — Part 5)

Missing	Needed for	Where it lands
Staff compensation (monthly salary, per-staff regular-hours override)	Payout estimation in money terms	HR app (R-4)
`attendance_regularization_request`	Staff-initiated corrections with approval	HR app (R-3) — interim flow is admin-edit (§2.6)
Approval-manager mapping per staff	Routing regularization requests	HR app (R-3); `staff_absence_record`'s approval fields prove the pattern in-house
Payroll snapshot / period lock	Freezing numbers money was paid against	R-5; `staff_attendance_summary.is_finalized` is the starting point

---

PART 2 — Build Plan: Matching the Existing Web System in the New Project

Goal: the new project reproduces the staff-attendance experience shown in the reference screenshots — same data, same behaviors — on the existing schema and (where present) existing endpoints. The full screen-level breakdown is Part 3; this part covers the data, rules, and platform work behind it.

2.1 Overtime calculation rules (parity semantics, written down once)

Parity keeps the existing pipeline — `staff_attendance_summary` holds the numbers — but we **freeze the formula in this doc** so web, mobile, and the recompute function agree:

```
For one staff, one date (school-local, via schools.time_zone):
  sessions      = non-deleted staff_attendance_record rows for that date, ordered by signed_in_at
  worked_min    = Σ (signed_out_at − signed_in_at)            // open records contribute 0 until closed
  break_min     = (last sign-out − first sign-in) − worked_min // gaps = breaks (unpaid by construction)

  config        = active staff_overtime_config for school
  if calculation_method == DAILY:
      regular_min  = min(worked_min, daily_regular_hours × 60)
      overtime_min = max(0, worked_min − daily_regular_hours × 60)
  if calculation_method == WEEKLY:
      weekly aggregation against weekly_regular_hours (existing semantics preserved — assumption A4)

Invariants (PM spec §2/§3):
  regular_min ≥ 0;  overtime_min ≥ 0
  overtime_min ≤ worked_min                  // overtime can never exceed total attended time
  total = regular_min + overtime_min = worked_min

Range totals (cards / summary section) = Σ over summary rows in the selected range.
```

Recalculation trigger (PM spec, Gherkin 2): the regular/OT recompute function runs after **every sign-out entry** — regardless of method (GPS/QR/PIN/APP/WEB), device, or which allowed user performed it — and after every admin edit/delete and auto sign-out. There is never a moment where a closed punch exists without an up-to-date summary.

Once the §1.3 columns land, grace/rounding apply at the marked points (`overtime_min` thresholded by `ot_min_daily_minutes` , floored to `ot_rounding_minutes`) — display-compatible, additive, default-off (grace 0 reproduces current behavior).

Validation gate before release: recompute one full month for pilot staff (e.g. the June reference data: 8 attended days, 64h regular + 11h 14m OT, per-day rows like Jun 11 = 8h + 16m OT with 1h 11m break) and diff against the old web system's numbers. Zero unexplained mismatches = parity proven.

2.2 Field-level spec (from PM product spec §3)

Field	Type	Validation
Regular Hours	Numeric (minutes internally; rendered `Xh Ym`)	≥ 0; cannot exceed total attended time
Overtime Hours	Numeric	≥ 0; derived from threshold; **cannot exceed total attended time**
Total Hours	Numeric	= Regular + Overtime (= attended time)
Break Minutes	Numeric	≥ 0; gaps between sessions; `–` rendered when 0/none
Date Range	Date pair	start ≤ end; **max range = 90 days**; presets Week / Month
Daily threshold (config)	Numeric hours	> 0; e.g. 8
Weekly threshold (config)	Numeric hours, optional	> 0 when set; ≥ daily threshold × 1
Sign-in / Sign-out edit	Date + time	sign-in < sign-out; sign-out ≤ now (+ sane bound); no overlap with the staff's other sessions that day

2.3 API work (new project)

Endpoint	Purpose	Status
`GET /staff-attendance/view/daily`	Daily roster (session rows) + filters + pagination	Exists — port/align

Endpoint	Purpose	Status
`GET /staff-attendance/view/custom`	Date-range view	Exists — port/align
`GET /staff-attendance/view/absent/toggle`, absent endpoints	Absent lists / Show Absent merge	Exists
`GET /staff-attendance/expected/summary/card`	Total/Expected staff cards	Exists
`GET /staff-attendance/profile`	Staff detail: cards + Separate rows + Summarised rows (`{summary, details[]}` per PM spec §6)	Exists — extend with regular/OT/break fields
`POST /staff-attendance-records/sign-in` , `/sign-out`	Create Attendance (admin-initiated)	Exists
`PUT /staff-attendance-records/daily/{rowCode}`	Edit sign in-out (+ `version` , audit, recompute)	Exists — harden (§2.6)
`DELETE /staff-attendance-records/daily/{rowCode}`	Delete record (soft) (+ audit, recompute)	Exists — harden
`POST /staff-attendance-records/transfer`	Classroom transfer	Exists
`POST /staff-absence-records/mark-absent`	Mark absent (single/bulk)	Exists
`POST /attendance/overtime-rules` (PM spec §6)	Admin sets daily/weekly threshold → upsert `staff_overtime_config` ; fires range recompute	**New** (assumption A5)
Report generation via `generated_reports`	Export: summary + detailed breakdown	Exists — add Excel & PDF formats (PM spec; current UI shows CSV)

Non-functional (PM spec AC6/AC7): profile + 30-day range loads **< 500 ms p95** (indexed `(staff_code, attendance_date)` reads + summary rows — comfortably within budget); every fetch failure renders *"Unable to load attendance data. Retry."* with a working retry.

2.4 Mobile implementation (this repo)

Follows the existing clean architecture (`lib/{domain,data,presentation}/features/attendance/`); most list/detail surfaces already exist here and need contract alignment, not rebuilds.

Layer	Work
Entities	Extend `StaffAttendanceDailyItem` family with `regularMinutes` , `overtimeMinutes` , `breakMinutes` , `signedOutMethod` (for the System badge). Add `StaffAttendanceRangeSummary` (cards + per-date rows for Summarised view).
Use-cases	`GetStaffAttendanceRangeSummaryUseCase` (Summarised), reuse existing daily/custom/absent/profile use-cases; edit/delete use-cases gain `version` pass-through.
Screens	Per the Part 3 inventory: staff list (3 tabs), staff detail with Summarised ⇄ Separate toggle, edit dialog, create-attendance / mark-absent / transfer forms.
Caching	Range summaries via existing Isar `CacheManager` , TTL + replace-on-fetch; invalidate on local edit/delete.

2.5 Web admin (new project)

Rebuild the screens in the Part 3 inventory against the same endpoints, plus the System badge and corrections hardening from Part 4.

2.6 Admin correction flow (PM-agreed interim for regularization)

Staff who spots a wrong/missing punch **contacts the admin**; the admin uses the Edit dialog (M-1). We harden this flow:

1. **Audit every correction** — `attendance_settings_audit_log` row with old/new values, actor, timestamp (§1.3). Disputes resolve with evidence.
2. **Optimistic locking** — `version` check; stale edit → 409 → UI prompts refresh.
3. **Instant recompute** — the staff-day summary recomputes on save; numbers are correct the moment the admin hits Save.
4. **Findability** — the System badge + "needs attention" count (Part 4) means admins can *see* which records likely need correction instead of relying on staff memory.

This is deliberately the simplest credible version of correction. Its limits (admin bottleneck, no staff self-service, no approval trail by the staff's manager) are exactly what R-3 in Part 5 addresses.

PART 3 — Complete Screen & Feature Inventory (Review Checklist)

> **Purpose:** founder & PM tick through this part to confirm nothing is missed. Every screen, component, pop-up, state, and behavior is listed with a source marker: ✅ = verified from screenshots / PM spec / codebase · ⚠ = built on a stated assumption (codebase/schema-derived), to be validated during build — all collected in Appendix B.
>
> Counts: **4 screens** (S-1…S-4, where S-1 has 3 tabs and S-2 has 3 card-driven views) · **9 pop-ups/modals** (M-1…M-9) · shared component kit (C-1…C-10) · global states (ST-1…ST-4).

3.0 Shared component kit (used across screens)

ID	Component	Behavior
C-1	Top bar	School name, messages icon + unread badge, notifications icon + badge, profile avatar + name + role + dropdown. ✅
C-2	Breadcrumb	`Home › Staff Attendance [› {Staff Name}]` ; segments clickable. ✅
C-3	Refresh button	Re-fetches current view with active filters; spinner while loading. ✅
C-4	Search input	Debounced (~300 ms) search by staff name/code; clears with Clear All. ✅
C-5	Dropdown filters	Select Classroom, Select Designation — single/multi select (⚠ confirm multi), resets page to 1 on change. ✅
C-6	Date / date-range picker	Single date (Daily tab) or range (profile/custom). Verified: pop-over with single month calendar, ‹ › month navigation, range selection (M-8). Presets **Week / Month** and **max range 90 days** are **PM-spec additions** to be built on top of the existing picker, with inline validation on exceeding the cap. ✅
C-7	Clear All	Resets search + all filters + date to default (today / current month). ✅
C-8	Data table	Sortable headers (Staff Name, Classroom, Date — sort icon toggles asc/desc), zebra rows, row-level Action `⋯` where applicable. **Rows with overtime > 0 are highlighted** (color/icon — PM spec AC8). ✅
C-9	Pagination bar	“Showing X to Y of Z entries”, first/prev/numbered/next/last, page-size selector (10/25/50 ⚠ confirm options). ✅
C-10	Toggle switches	`Show Absent` , `Separate ⇄ Summarised` — styled switch, label states. ✅

3.1 S-1 · Staff Attendance List (admin)

Route: `Staff Attendance` . Tab strip: **Daily | Custom | Absent**. ✅

S-1a · Daily tab ✅

Area	Components & functionality
Summary cards	**Total Staff** — count + progress bar + legend `Active (n)` / `Upcoming (n)` . **Expected Staff** — count + bar + legend `Signed-in (n)` / `Signed-out (n)` / `Absent (n)` / `Not Marked (n)` . Both from `expected/summary/card` ; refresh on filter/date change.
Actions row	C-3 Refresh · **+ Create Attendance** button → M-4 · **Show Absent** toggle → merges absent staff rows into the table (absence rows render with marked-absent time/by/note per the verified S-2c pattern).

Area	Components & functionality
Filter row	C-4 Search Staff · C-5 Select Classroom · C-6 single Date (default today) · C-5 Select Designation · C-7 Clear All.
Table	Columns: Staff Name (sortable, avatar/initials) · Expected Today? (Yes/No) · Classroom (sortable) · Signed-in time & by · Signed-out time & by · Attended Time · Action ⋯ → M-7. **One row per session** — a staff with 3 sessions appears 3× (verified: Spandana Marri 3 rows). "By System" renders when `signed_out_method='SYSTEM'` . Staff-name click → S-2 profile (A11).
Footer	C-9 pagination.

S-1b · Custom tab ⚠ (assumption A6)

Expected per codebase (`staff_attendance_custom_view.dart` + `view/custom` endpoint): date-**range** picker instead of single date, same filters, session rows across the range with Date column, same Action menu, pagination.

S-1c · Absent tab ⚠ (assumption A6)

Expected per codebase (`staff_attendance_absent_view.dart` + absence endpoints): date/range filter, table of `staff_absence_record` rows — Staff Name, Date(s), Absence Type (Full/Half), Reason, Marked By/At — plus **Mark Absent** action → M-5; row Edit/Delete per the verified S-2c pattern (M-2/M-3).

3.2 S-2 · Staff Attendance Profile (admin view of one staff)

Route: `Staff Attendance › {Staff Name}` . ✅

Area	Components & functionality
Controls row	C-6 date-range picker (defaults to current month, e.g. `06/01/2026 – 06/30/2026`) · C-3 Refresh · **Separate** ⇄ **Summarised** toggle (C-10) switching the attendance listing between S-2b and S-2a.
Header card	Staff photo **or initials avatar** (e.g. `DV`) with a status dot (red dot observed — assumed "absent/not signed in today" indicator ⚠), name, classroom/designation (e.g. "Office", "4-Yellow-D").
Stat cards = **the "3 tabs"** ✅	Three clickable stat cards, recomputed for the selected range; the **active card gets a highlighted border** and drives the lower table (verified across screenshots): ① **Attended Days** (count, green ring) → attendance listing (S-2a/S-2b per toggle) · ② **Attended Time** (total e.g. `71h 3m` , stacked bar + legend `Regular (64h)` / `Overtime (7h 3m)` , OT in alert color) → Summarised OT breakdown (S-2a) · ③ **Absent Days** (count, red ring) → absence records view (S-2c). Exact coupling between card selection and the Separate/Summarised + Show Absent toggles to be confirmed during build (⚠).

S-2a · Summarised view ✅ (per-day rollup — "the overtime screen")

Area	Components & functionality
Actions	**Export** button → M-9. Current UI: CSV. PM spec requires **Excel & PDF** — build supports all three via `generated_reports` (`format` field).
Table	Columns: Date (sortable) · Regular Hours · **Overtime Hours** (header + values in alert color) · Total Hours · Break Minutes (– when none). One row per attended day from `staff_attendance_summary` . OT rows highlighted (AC8). Totals row / summary section shows cumulative Regular, Overtime, Total for the range (AC2).
Footer	C-9 pagination.

S-2b · Separate view ✅ (session-level rows)

Area	Components & functionality
Actions	**Show Absent** toggle (C-10) — switches/merges absence rows into the listing (verified ON-state rendering in S-2c).
Table	Columns: Date (sortable) · Classroom · Sign In · By · Sign Out · By · Attended Time · Action ⋯ → M-7. One row per session; multiple rows per date stack chronologically (verified: 6 sessions on Jun 11). `By = System` for auto-closes → rendered as badge (Part 4).
Footer	C-9 pagination.

S-2c · Absent records view ✅ (Absent Days card active / Show Absent ON)

Area	Components & functionality
Table	Columns: Date (sortable) · Classroom (color chip avatar + name, e.g. `4Y · 4-Yellow-D`) · Marked Absent (date + time, e.g. `2026-06-11, 09:14 AM`) · By (who marked, e.g. `Spandana Marri`) · Absent Note · Action ⋯ → **Edit** (M-2 Edit Absent) / **Delete** (M-3). Backed by `staff_absence_record` (`marked_absent_at/by_code`, `absence_note`).
Footer	C-9 pagination ("Showing 1 to 2 of 2 entries").

3.3 S-3 · Staff's own Attendance Profile (staff persona — PM spec user journey) ⚠

Jessica (Staff) logs in → views **her own** Regular/OT breakdown → filters by date range. Same composition as S-2 minus admin powers: no Edit/Delete in Action menu (or no Action column), no Create Attendance; export allowed. Role gate: staff sees self only; admins see anyone. Assumption A9: same screen, role-gated read-only (not a separate build).

3.4 S-4 · Overtime Rules Settings (admin) ⚠ (new screen per PM spec §2/§6 — assumption A5)

Area	Components & functionality
Form	**Daily threshold** (hours, e.g. 8) · **Weekly threshold** (optional) · maps to `staff_overtime_config` (`calculation_method`, `daily_regular_hours`, `weekly_regular_hours`). Part 4 adds the grace/rounding knobs here later (default hidden/advanced).
Behavior	Save → `POST /attendance/overtime-rules` → success toast → **recompute affected summaries** so OT reflects the new threshold (PM QA matrix: "Configure overtime threshold → overtime recalculated correctly"). Recompute scope on rule change per assumption A5: the current open (unfinalized) period only.
Placement	Per assumption A5: school Settings section; org-level default with per-school override considered a future option.

3.5 Pop-ups & modals

ID	Pop-up	Trigger	Contents & behavior
M-1	**Edit Sign In-Out** ✅	Action ⋯ → Edit on attendance rows (S-1a, S-2b)	Title + close ✕. Sections: **Sign In** (Date, Time) and **Sign Out** (Date, Time). **Save** validates per §2.2 (in < out, ≤ now, no overlap), sends `PUT` with `version`; success → toast + row + cards + summary refresh; 409 → "Record was changed by someone else — refresh and retry."
M-2	**Edit Absent** ✅	Action ⋯ → Edit on absence rows (S-2c)	Title "Edit Absent" + close ✕. Fields: **Absent Date** (date input) · **Absent Note** (multiline text). Buttons: **Cancel** / **Submit**. Updates `staff_absence_record`; refreshes Absent Days card + table.
M-3	**Delete confirmation** ⚠	Action ⋯ → Delete (attendance & absence rows)	Standard confirm dialog assumed (record summary + warning) → soft delete → audit row → recompute → toast + refresh. Copy/reason-required to confirm during build.
M-4	**Create Attendance** ⚠	+ Create Attendance (S-1a)	Per codebase: choose staff (from `sign-in/form/list` / `sign-out/form/list`), sign-in or sign-out action, classroom, time; method recorded as APP/WEB. Exact field layout assumed from mobile forms.
M-5	**Mark Absent** ⚠	Absent tab / action	Per codebase + schema: staff (single/bulk via `absence_batch_code`), date or from–to range, absence type (Full/Half), reason (Sick/Leave/Other), note.
M-6	**Transfer classroom** ⚠	Action menu (if exposed on web)	From/to classroom, time, reason → `staff_classroom_transfer`. Assumed mobile-only until confirmed.
M-7	**Action ⋯ menu** ✅	Row action cell	**Exactly two items verified: Edit, Delete.** On attendance rows → M-1/M-3; on absence rows → M-2/M-3. Hidden/disabled for non-admin roles.
M-8	**Date-range picker pop-over** ✅	C-6	Verified: single-month calendar pop-over, ‹ › month navigation, range select. **Build adds (PM spec):** Week/Month presets + 90-day cap with inline error.

ID	Pop-up	Trigger	Contents & behavior
M-9	**Export dialog / progress** ⚠	Export button (S-2a)	Format choice (CSV / Excel / PDF per PM spec), generation via `generated_reports`, download link; file contains **summary + detailed breakdown** (AC5). Instant vs. async generation decided by range size during build.

3.6 Global states & non-functional behaviors

ID	State	Behavior
ST-1	Loading	Skeleton rows in tables; spinner in cards; controls disabled during fetch.
ST-2	Empty	Friendly empty state per tab/view ("No attendance records for the selected range").
ST-3	Error	**"Unable to load attendance data. Retry."** with working Retry (PM AC7) on every fetch surface; mutation failures keep dialog open with inline error.
ST-4	Permission-gated	Create/Edit/Delete/Settings visible to School/Org Admin only; staff persona read-only self view (S-3). Salary-free build — no money shown anywhere in this phase.
NF-1	Performance	30-day profile loads **< 500 ms p95** (PM AC6).
NF-2	Accessibility	All fields/controls labeled for screen readers (PM QA matrix); OT highlight not color-only (icon + color, AC8).
NF-3	Timezone	All dates/times rendered in `schools.time_zone`.

3.7 Acceptance-criteria traceability (PM spec §4)

AC	Covered by
AC1 Regular & OT displayed separately per day	S-2a table
AC2 Summary shows cumulative Regular/OT/Total	S-2 header cards + S-2a summary section
AC3 OT auto-calculated from configured threshold	§2.1 formula + S-4 settings + recompute trigger
AC4 Date-range filter (Nov 01–30 → only those dates)	C-6 on S-1b/S-2
AC5 Export = accurate summary + detail	M-9
AC6 < 500 ms p95 (30-day)	NF-1, §2.3
AC7 Error + Retry message	ST-3
AC8 OT rows highlighted (color/icon)	C-8, S-2a, NF-2
AC9 OT ≤ total attended time	§2.1 invariants, §2.2 validations

---

PART 4 — Improvements Included in This Build (near-zero extra cost)

These ship with parity because they are days, not weeks, and they fix real correctness/visibility holes:

4.1 System sign-outs become visible

Auto-close stamps `signed_out_method = 'SYSTEM'`; both clients render a **"By System"** badge (the old web UI already shows "05:30 PM By System" — we make it a first-class, queryable signal rather than display text). *One line in the job + a badge.*

4.2 "Needs attention" count for admins

Per staff per range: count of system-closed days (`signed_out_method = 'SYSTEM'`) and never-signed-out days. Shown on the staff detail header and as a list filter. *A count query — but it converts the silent wrong-overtime problem into a visible, actionable one, and it's the bridge to the future regularization flow.*

4.3 Corrections that hold up

Audit trail + optimistic locking + recompute-on-edit (§2.6). *Three small backend hooks; the difference between "the admin changed something" and "we can prove what changed, by whom, and the numbers updated instantly."*

4.4 Grace & rounding knobs

The `staff_overtime_config` money columns (§1.3) with defaults that reproduce current behavior (grace = 0 until a school opts in). *Additive columns; unlocks per-location OT policy the day a school asks for it.*

PART 5 — Pendency: Improvement Roadmap for Founder Review

> Nothing below blocks the Part 2–3 build. Each item is a self-contained proposal: **what**, **builds on** (the reuse path from Part 1), and **impact on product goodwill**. Full technical designs for all of these exist in `STAFF_ATTENDANCE_OVERTIME_TECH_SPEC.md`.

R-1 — Retire the summary scheduler; compute attendance/OT on-demand from raw records

What: replace stored `staff_attendance_summary` reads with a calculation engine that derives daily/monthly numbers from `staff_attendance_record` at read time, cached per staff-month, invalidated on writes. The recompute-on-sign-out/edit hook from §2.1 is the first step of exactly this migration. **Builds on:** the frozen formula in §2.1 — same math, different trigger. **Goodwill impact:** numbers that are *never* stale — an edit reflects everywhere instantly, always. Server cost drops (compute only what is read, not all-staff-every-night). Engineering stops babysitting jobs. Customers experience it as "this product is never wrong about my hours," which is the entire reputation of an attendance product.

R-2 — Honest missing-punch handling (per-school policy)

What: extend §4.1 — schools choose `AUTO_CLOSE` (current behavior, visibly flagged) or `APPROVAL_BASED` (record stays open, excluded from confirmed totals, queued for correction). `attendance_settings.auto_sign_out_enabled` already is this toggle's foundation. **Goodwill impact:** staff who stay late for emergencies stop losing earned overtime to a fabricated 2:00 PM sign-out. One avoided incident of "the app stole my overtime" is worth more goodwill than any feature launch — and the inverse story spreads in staff rooms fast.

R-3 — Attendance regularization workflow (the HR application)

What: staff raise "I actually left at 3:30" requests from a Needs-Attention prompt; requests route to their approval manager (school admin fallback, org admin above school admins, topmost admin self-approved); approvals inbox; approved requests update the record + audit. The `staff_absence_record` approval fields (`requires_approval`, `approval_status`, `approved_by_code`) prove this pattern already works in our stack — we generalize it. **Goodwill impact:** removes the admin bottleneck of §2.6 and matches table-stakes HR products (Keka, Zoho People, greytHR) — closes the recurring sales objection. Managers finally *know* who misses punches and when people really leave. Staff get self-service dignity instead of chasing admins.

R-4 — Staff-visible estimated payout

What: staff compensation (monthly salary, per-staff hours override) lives in the HR app; rate = salary ÷ (working days × daily hours) using `childcare_schedules` for the calendar; staff see live month-to-date estimated payout (salary + OT × multiplier − provisional deduction). **Goodwill impact:** pay transparency is the strongest trust lever in workforce software. Staff check this screen daily — attendance compliance rises because every punch has a visible price. Most childcare competitors don't show staff their money; we would stand out on the strength of one screen.

R-5 — Payroll finalization: locked periods & snapshots

What: grow `staff_attendance_summary.is_finalized` into a real contract: admin resolves all flagged items with explicit choices (regularize / default to school end / approve full pay / deduct at 1.0×|0.5×), finalizes the month → immutable snapshot of numbers + decisions; edits to finalized periods rejected unless explicitly reopened (audited). **Goodwill impact:** the system never decides money silently, and paid history can never quietly change underneath anyone. This is what lets a founder say "automate payout" with a straight face — and what auditors and enterprise buyers ask for by name.

R-6 — Effective-dated configuration

What: versioned-by-date rows for `staff_overtime_config` and compensation, resolved as-of each attendance date — a multiplier change on June 15 computes June 1–14 at the old rate, provably. **Goodwill impact:** mid-month salary revisions

and policy changes — which happen at every customer — become correct by construction instead of "please change settings only on the 1st." The product earns the reputation of handling messy reality.

R-7 — Prevention & intelligence (geofence nudges, anomaly flags)

What: (a) we already store `signed_out_within_geofence` and GPS points — when a staff phone leaves campus with an open record: "You left at 3:32 PM — sign out?" (b) the engine flags impossible days (>16h), OT spikes vs the staff's own baseline, chronic missed punches. **Goodwill impact:** missed punches shrink at the source, so R-3's correction volume trends toward zero — the product looks *smart* in demos and protects the customer's money before payday. This is differentiation competitors can't copy with a settings page.

Suggested sequencing

Wave	Items	Theme
Now	Parts 2–3 + Part 4	Parity with honest data
Next	R-1, R-2	Correctness as the product
HR app v1	R-3, R-4	Self-service + transparency
HR app v2	R-5, R-6	Payroll-grade trust
Differentiators	R-7	Prevention & intelligence

Appendix A — Edge cases handled in the parity build

Case	Handling
Multiple sessions/day	Sum sessions; gaps = `total_break_minutes` (§2.1).
Open record (no sign-out yet)	Contributes 0 worked minutes until closed; counted in needs-attention (§4.2).
Auto-closed record	Counted up to auto-close time; visibly `SYSTEM` (§4.1).
Edit after summary job ran	Recompute-on-mutation (§1.3/§2.1) — summary never lags a correction or sign-out.
Concurrent edits	`version` optimistic lock → 409 (§2.6).
Deleted record	Soft delete; summary recomputed; audit row written.
Timezone	All date bucketing via `schools.time_zone`; timestamps stored UTC.
Transfers mid-day	Sessions split per classroom via `staff_classroom_transfer`; durations unaffected.
Half-day / date-range absences	`staff_absence_record` `absence_type` + from/to range respected in absent counts.
OT threshold changed	Recompute scope per S-4 decision (A5); OT never exceeds attended time regardless (AC9).
Range > 90 days	Blocked client-side + server-side with validation message (§2.2).

Appendix B — Assumptions & remaining confirmations (non-blocking; validate during build)

Resolved by the second screenshot set: stat cards act as the 3 tabs with selected-state border (S-2), absent records view columns (S-2c), Edit Absent modal (M-2), Action `⋯` menu = Edit + Delete (M-7), date-picker pop-over behavior (M-8).

#	Item	Assumption applied in this doc	Validate at
A1	`signed_out_method` accepting `'SYSTEM'`	Column is unconstrained varchar; current auto-close job leaves a marker we can backfill from	§4.1 backfill, sprint 1
A2	`staff_attendance_summary.summary_type` values	DAILY rows only; range cards aggregate over daily rows	S-2 cards, sprint 1
A3	Microservice split unchanged in new project	`schools.time_zone` / `childcare_schedules` remain cross-service reads	§2.3

#	Item	Assumption applied in this doc	Validate at
A4	Weekly OT semantics (`calculation_method = WEEKLY`)	Existing behavior preserved as-is; week = school-local calendar week	§2.1 freeze, before formula sign-off
A5	Overtime Rules screen (S-4)	New screen under school Settings; threshold change recomputes the current open (unfinalized) period only	S-4 design review
A6	Custom tab (S-1b) & Absent tab (S-1c) layouts	Mirror codebase views: range picker + session rows; absence rows + Mark Absent action	First UI review
A7	Create Attendance (M-4) / Mark Absent (M-5) field sets	Match the existing mobile forms in this repo	First UI review
A8	Delete confirmation (M-3)	Standard confirm; no reason field	First UI review
A9	Staff self view (S-3)	Same S-2 screen, role-gated read-only (no Action column, no Create)	RBAC review
A10	Export	CSV kept + Excel & PDF added (PM spec); Summarised-view columns; async generation for ranges > 31 days	M-9 build; column set with finance
A11	Table details	Page sizes 10/25/50; classroom/designation filters single-select; Daily-tab staff-name click opens profile	First UI review
A12	Header-card red status dot (S-2)	“Absent / not signed in today” indicator	First UI review